

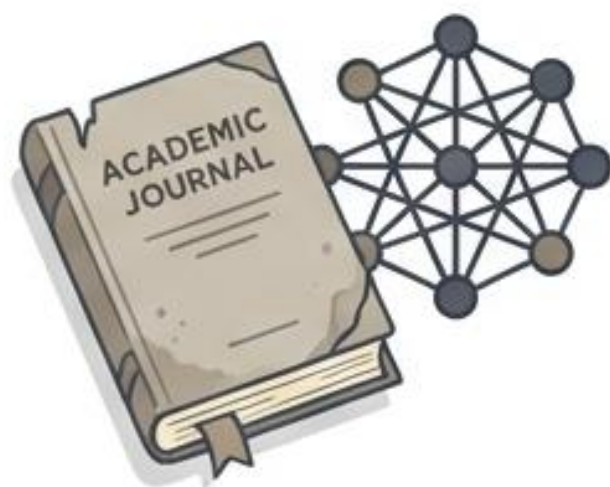


Graph Databases

Building and Querying Knowledge Graphs
with Neo4j & Cypher

dr Zbigniew Galar

The Past



Knowledge graphs were historically theoretical concepts, largely confined to academia and early Semantic Web research.



The Present



Modern graph databases (like Neo4j) have moved knowledge graphs into practical, high-performance production systems. Technologies like AuraDB offer cloud-native services ready for enterprise scale.

**From semantic theory to
high-performance data science.**

Humane Design

Cypher's primary design goal is to be highly readable and predominantly visual, matching data structures with ASCII art conventions.

Visual Canvas



Code Editor

```
1 (:Person {name:'Rosa'})-[:LIVES_IN]->
  (:Place {city:'Berlin', country:'DE'})
```

Industry Standard

Originally created by Neo4j, Cypher is currently being standardized by ISO as GQL (Graph Query Language — "SQL for Graphs").

CREATE (:Person {name:'Rosa'})-[:LIVES_IN {since:2020}]->(:Place {city:'Berlin'})

Nodes

The entities in the graph (drawn as circles).

Labels

Group nodes by role (e.g., Person). Powers backend indexing for faster performance.

Relationships

Define the connection and mandatory directionality (e.g., Rosa lives in Berlin).

Properties

Key-value pairs stored directly on nodes or relationships to add rich context.

Terminal Input Box

```
CREATE (:Person {name:'Rosa'})-[:LIVES_IN {since:2020}]->(:Place {city:'Berlin', country:'DE'})
```



Execution

Canvas Visualizer



Key Rules

- **Directionality is Mandatory:** When storing data, the direction of a relationship must be specified to create a high-fidelity model.
- **High Fidelity:** If you know something about your domain, put it in your knowledge graph.

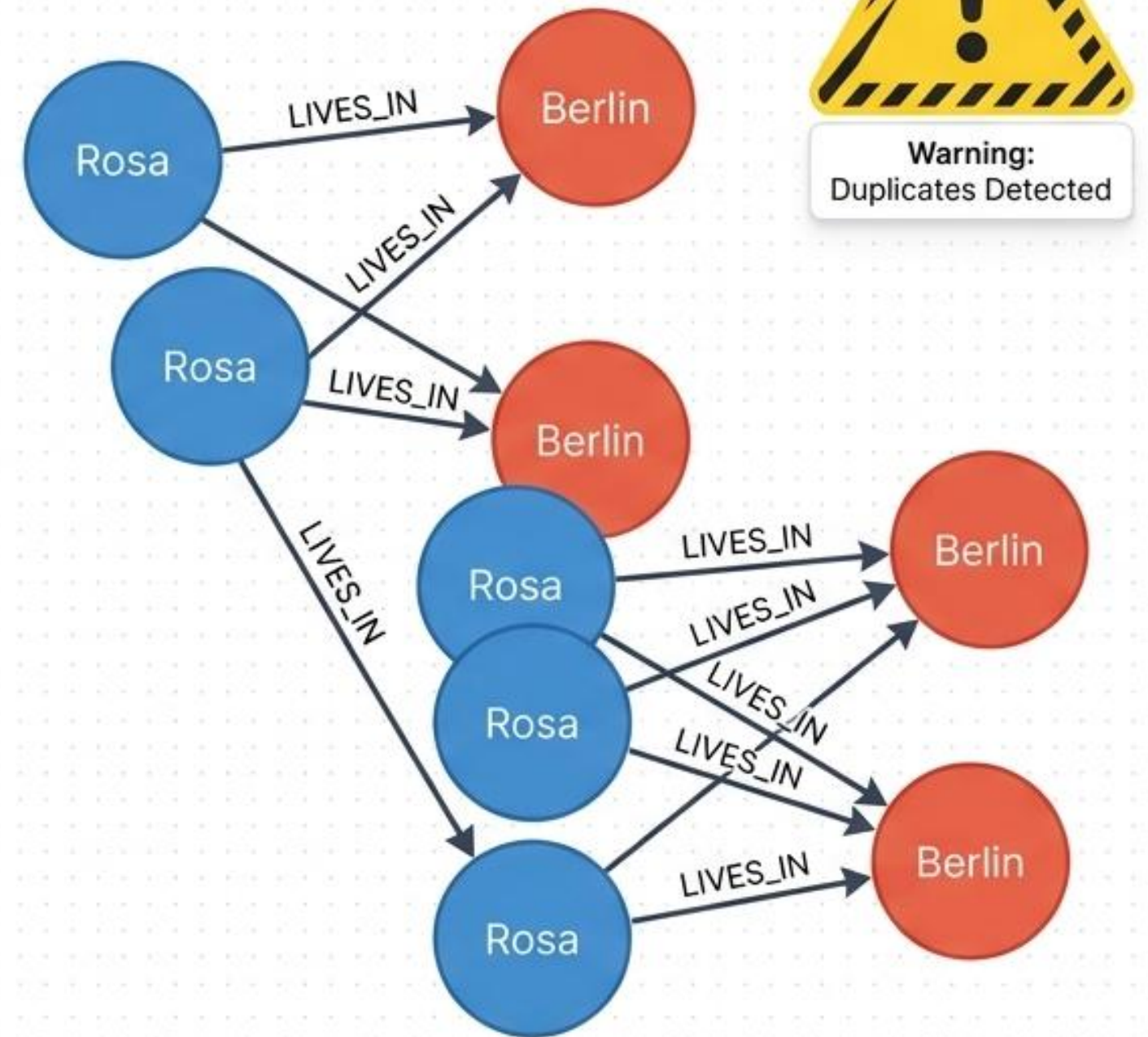
The Duplicate Dilemma

CREATE is a blunt instrument. It always generates new records and never checks for **existing data**. Running the **same script twice** creates unwanted **duplicates**.

```
MATCH (n) DETACH DELETE n
```

DETACH DELETE is the nuclear option for clean-up. It safely deletes all nodes and their attached relationships, preventing dangling connections.

Canvas Visualizer



CREATE

(The Naive Approach)

Behavior: Blindly inserts data.

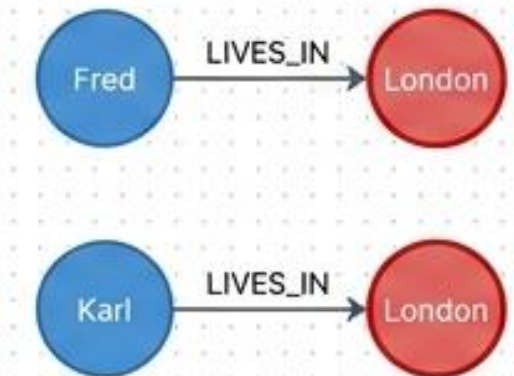
Result: Guaranteed duplicates if run repeatedly.

MERGE

(The Match or Create Approach)

Behavior: Evaluates the entirety of a supplied pattern. If it doesn't exist perfectly, it creates it.

The Trap: It never partially matches. If a London node exists, merging a new person living there with a single line of code will recreate the entire pattern.



Best Practice: Never MERGE complex paths all at once. Deconstruct operations into individual node and relationship merges.

The Composite Constraint

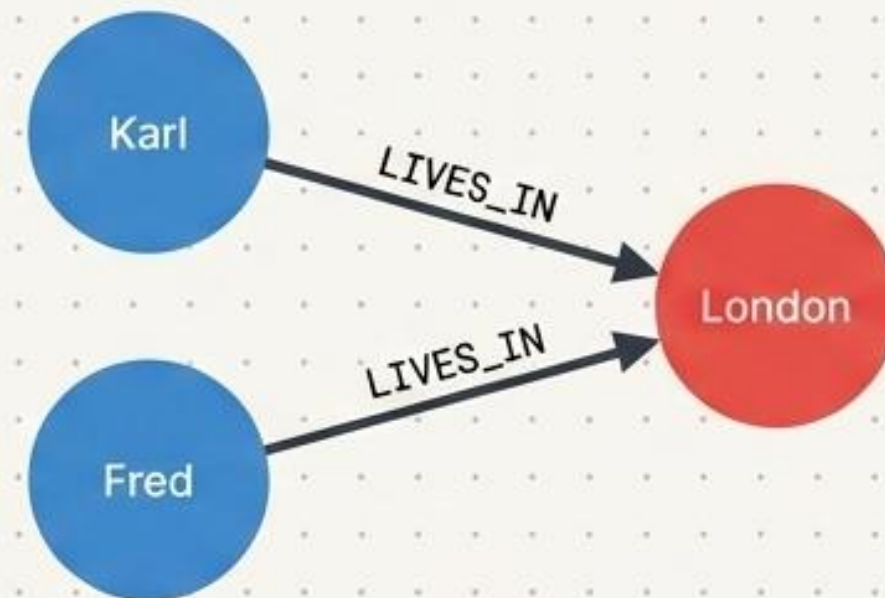


```
CREATE CONSTRAINT no_duplicate_cities
FOR (p:Place)
  REQUIRE (p.country, p.city) IS NODE KEY
```

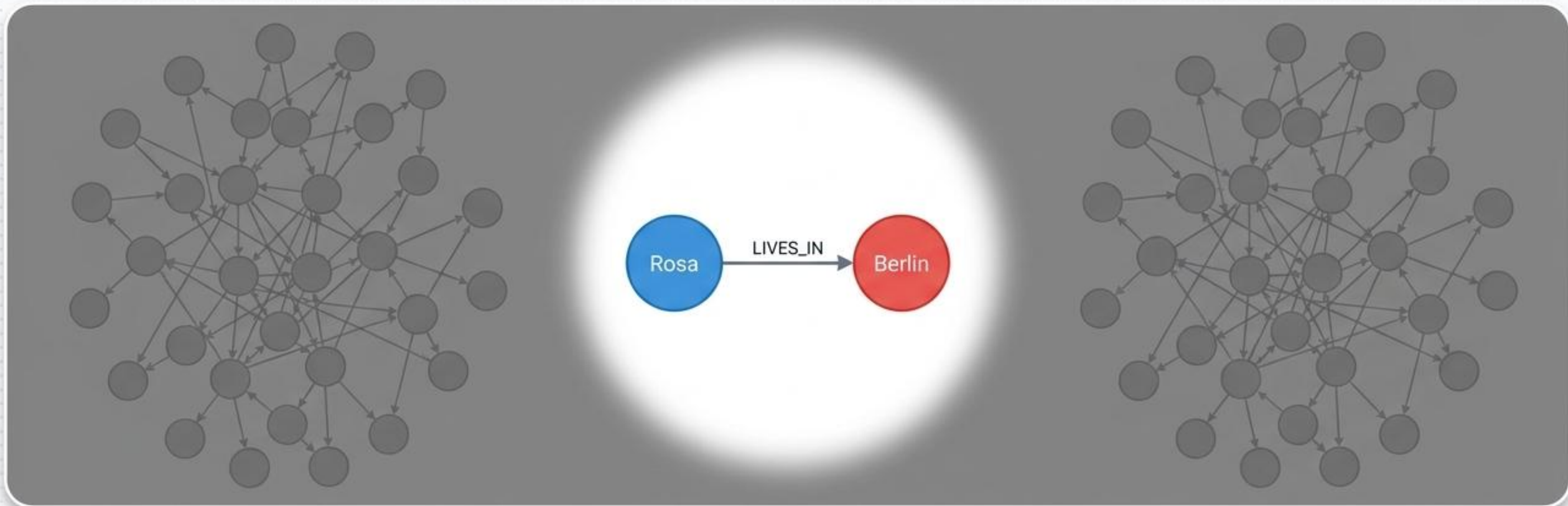
Allows 'London, UK' and 'London, Ontario' to coexist, but structurally rejects any transaction trying to create a duplicate 'London, UK'.

Deconstructed MERGE

```
MERGE (london:Place {city:'London', country:'UK'})
// Safely finds or makes the city.
MERGE (fred:Person {name:'Fred'})
// Safely finds or makes the person.
MERGE (fred)-[:LIVES_IN]->(london)
// Safely connects them.
```



Result: Executed atomically, guaranteeing zero duplicates.



The Engine

MATCH compares user-defined ASCII patterns against the underlying database topology.

Knowns vs. Unknowns

```
MATCH (p:Person)-[:LIVES_IN]->(Place {city:'Berlin'})
```

You provide known anchor points (Place: Berlin) and let the database bind the unknowns to an unbound variable (p).

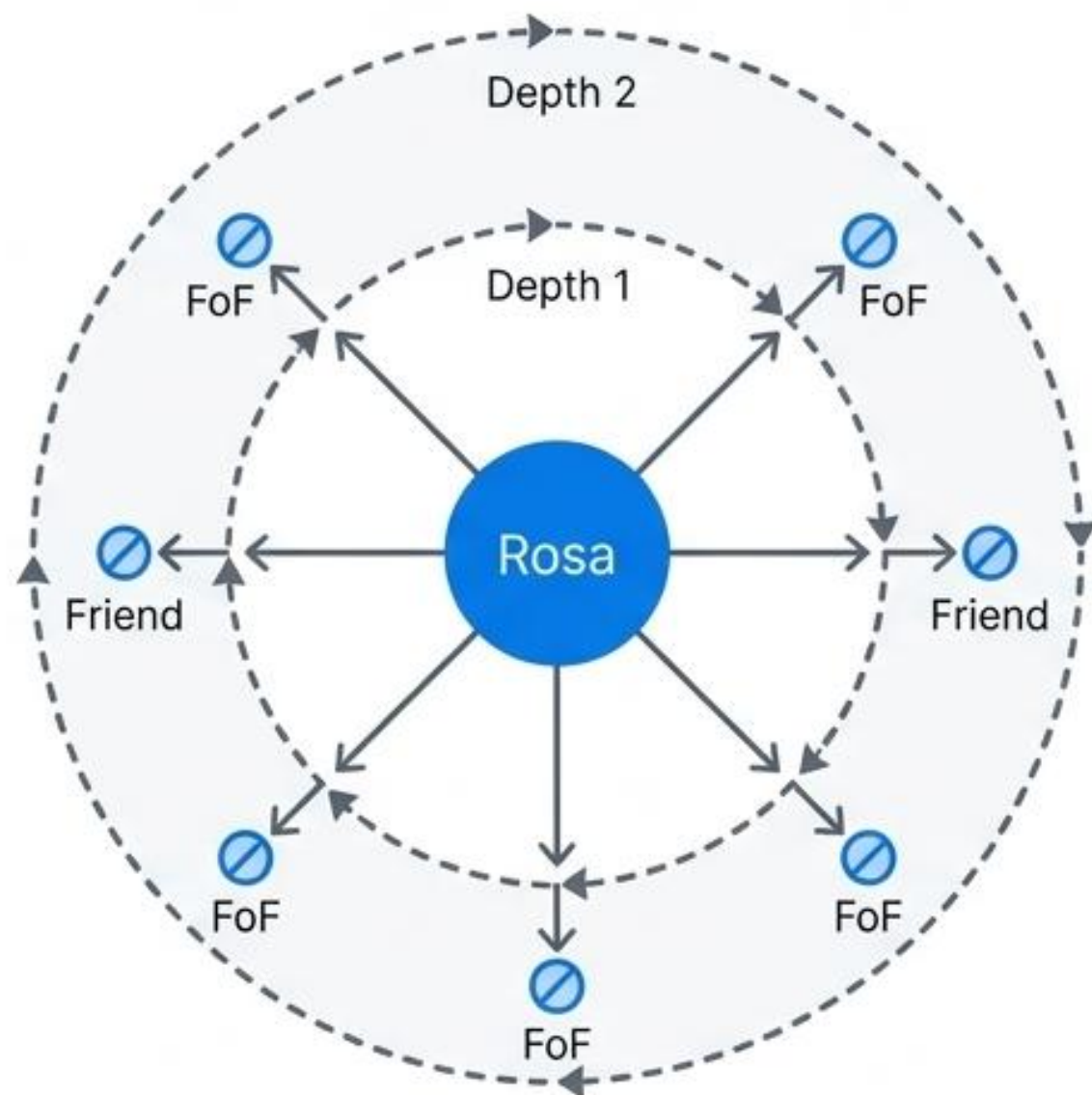
The Output

```
RETURN (p)
```

Outputs the bound variables (the matched results) to the user.

The Digital Canvas

Ripple Effect



Graph Local Queries

Queries anchored to a specific, known starting node.

```
MATCH (:Person {name: 'Rosa'})-[:FRIEND*2..2]->(fof:Person)
```

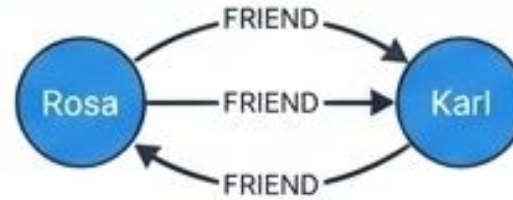
Syntax `*2..2` traverses the graph to find a friend-of-a-friend exactly two hops away.

The Graph Performance Advantage

Unlike relational databases that suffer from recursive join-bombs, Neo4j traverses relationships at a constant cost. Query latency is proportional to the subgraph searched, not the total database size.

The Challenge: Circular Loops

```
MATCH (rosa:Person {name:'Rosa'})-[:FRIEND+2..2]->(fof:Person)
RETURN fof
```



Raw graph traversal can return the starting node (Rosa) or an intermediate friend (Karl) as the "fof" result, creating circular loops.

WHERE rosa <> fof

Boolean predicates enrich the pattern matching to ensure the starting node (rosa) does not equal the ending node (fof), filtering out circular results.

The Refined Result

Fred

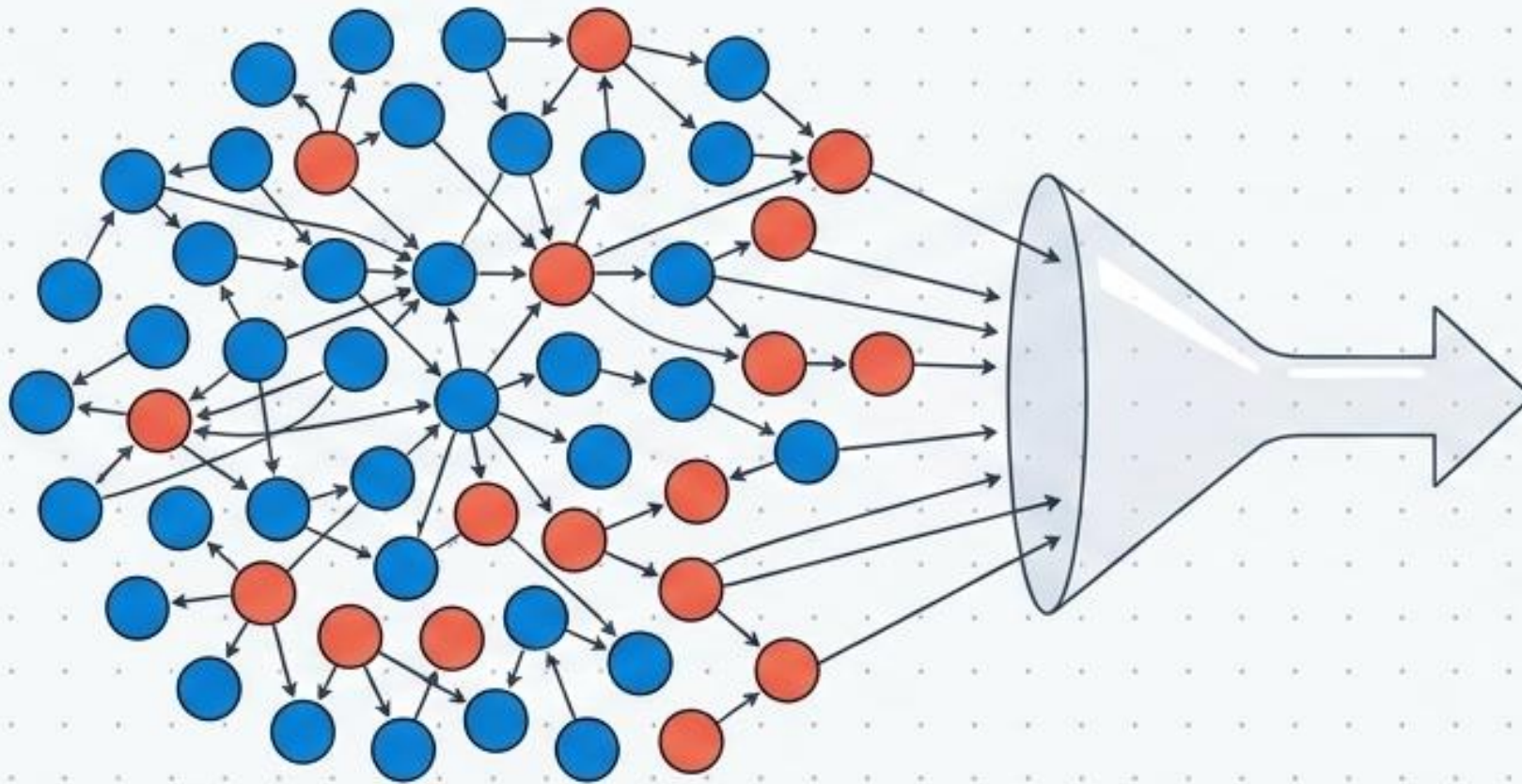
A single, clean output graphic representing the actual friend of a friend.

Other Predicate Capabilities

- String matching:
`STARTS WITH`, `CONTAINS`
- List operations:
`IN ['Rosa', 'Karl']`
- Pattern exclusion:
`NOT (p)-[:KNOWS]->()`

Graph Global Queries

Queries that do not have a specific starting anchor. They process large portions of the entire graph to calculate metrics, aggregations, or rankings.



place	rels
{country:"UK", city:"London"}	2
{country:"DE", city:"Berlin"}	1

```
RETURN p AS place, count(1) AS rels  
ORDER BY rels DESC
```

Utilizes SQL-familiar concepts like aliases (AS), aggregations (count()), and sorting (ORDER BY).

Pro-Tip

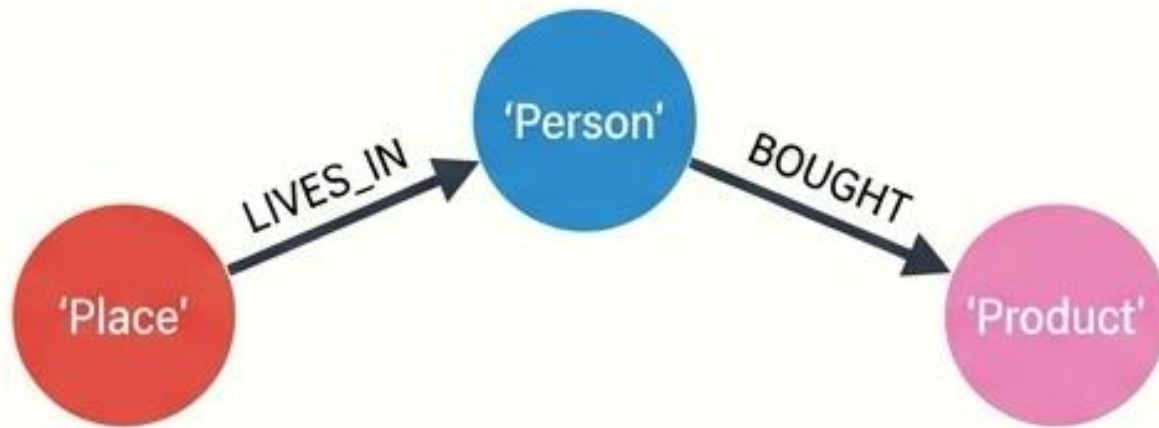
Always provide node labels in patterns (e.g., (:Person)). Explicit labels help the query planner optimize the search space and reduce latency.

Toolbox

Database Procedures (CALL)

Executed tasks for meta-graph operations.

```
CALL db.schema.visualization()
```



Reveals the underlying meta-schema of the entire knowledge graph.

The APOC Library

Awesome Procedures on Cypher — Neo4j's massive standard library. Avoid reinventing the wheel for string manipulation, formatting, and complex math.

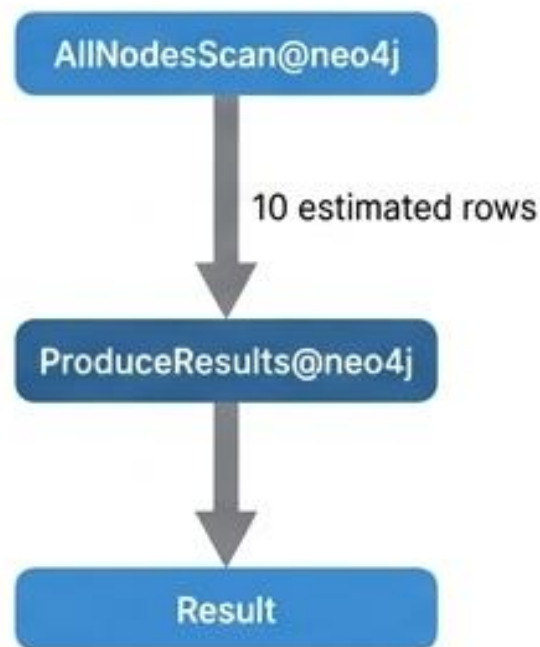
Inline Function Example

```
RETURN apoc.date.convert(datetime().epochSeconds, 'seconds', 'days')
```

Example: Converts epoch timestamps natively without complex logic.

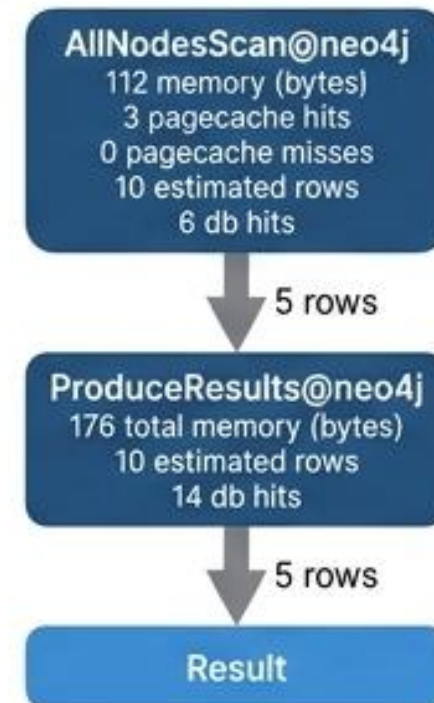
Diagnostic Matrix

EXPLAIN



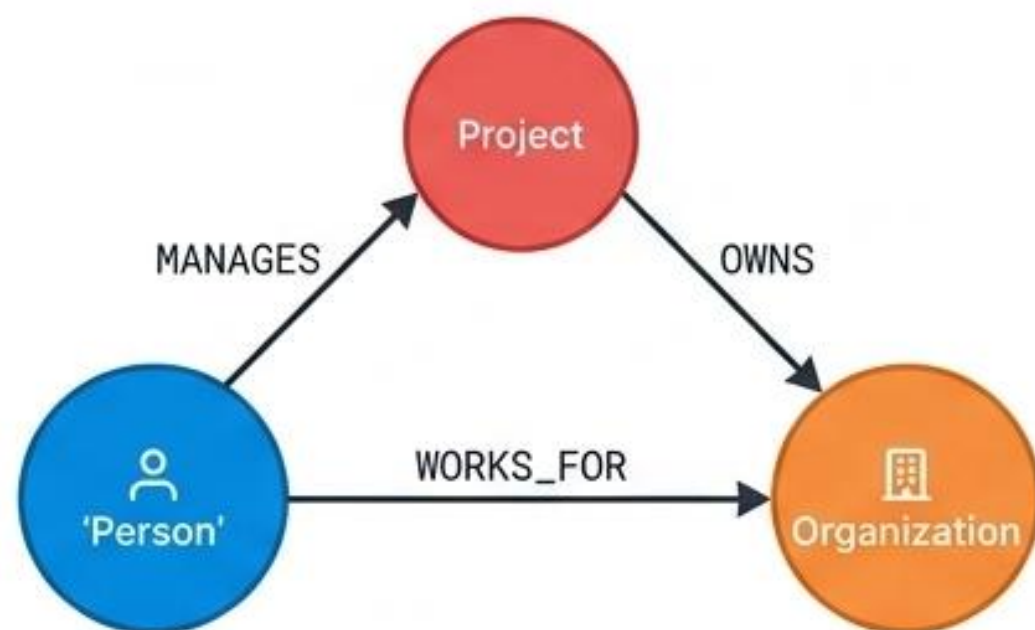
What it is:	The Theoretical Plan.
Action:	Shows how the query planner thinks it will execute without running it.
Use Case:	Visualize expected database hits before running a heavy query.

PROFILE



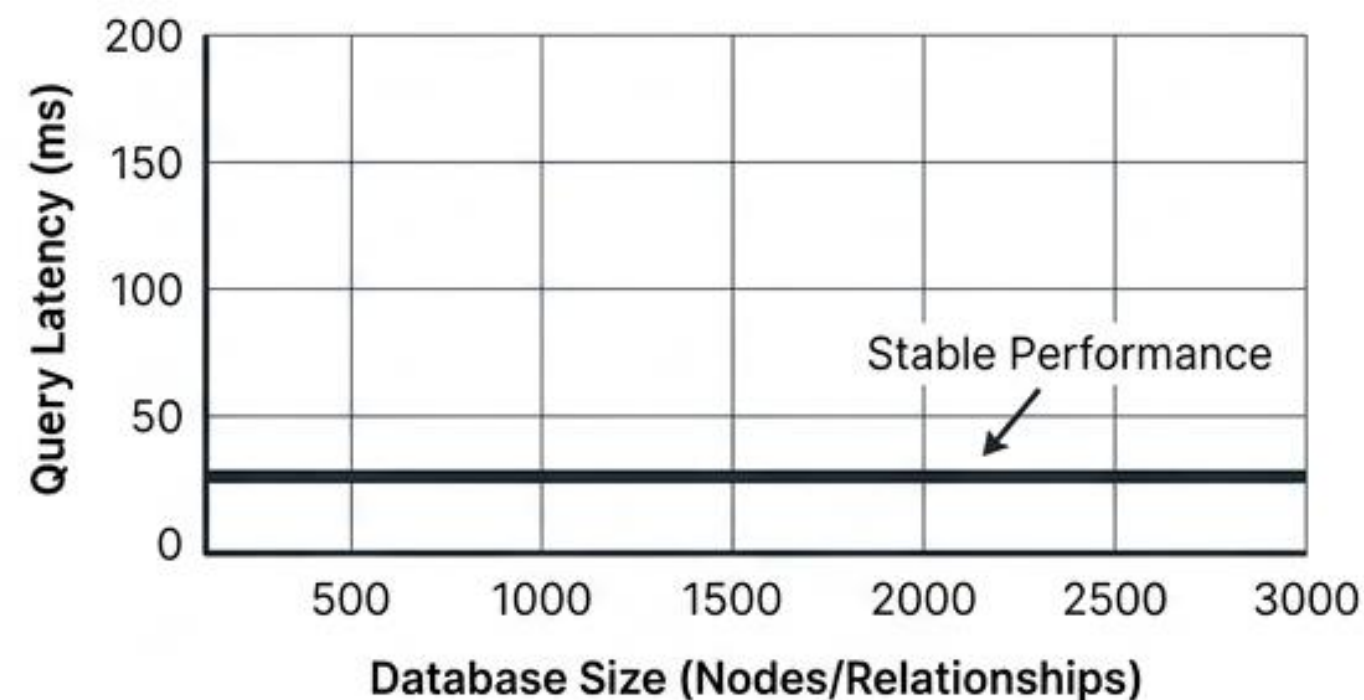
What it is:	The Runtime Reality.
Action:	Instruments the running query to capture actual execution metrics.
Use Case:	Track exact DB hits per operation, spot missing indexes, and tune latency.

Visual Fidelity (The Model)



Code equals diagram. If you know how your domain works in the real world, you can draw it. If you can draw it, you can query it. Cypher democratizes complex data access.

Unpenalized Scale (The Performance)



Graph databases eliminate rigid relational schemas and recursive join penalties. By traversing relationships at a constant cost, search depth can expand freely while performance remains perfectly stable.

Graph databases bridge the gap between human intuition and high-performance data processing.